

Versioning

COMPLETE LEARNING GUIDE

What You'll Learn

APIs change over time, but the clients consuming them often can't update at the same pace — a mobile app version out in the wild, a partner's integration that runs unattended for years — so a breaking change to an API can silently break every client still on the old contract. Versioning is how an API introduces breaking changes without breaking existing consumers: old and new contracts coexist for a transition period, giving clients time to migrate. This guide compares the three common versioning strategies (URI path, custom header, and query parameter), explains what actually counts as a 'breaking' change versus a safe additive one, and covers how to responsibly deprecate and eventually retire an old version rather than maintaining every version forever.

1. What Counts as a Breaking Change?

Not every API change requires a new version — only changes that would break a client written against the current contract. Adding a new optional field to a response, adding a new optional query parameter, or adding an entirely new endpoint are additive and safe: existing clients that don't know about the new field simply ignore it. Removing a field, renaming a field, changing a field's data type (a number becoming a string), changing the meaning of an existing status code, or making a previously optional request field required are all breaking changes that demand either a new version or a carefully staged migration.

- Safe (non-breaking): new optional response field, new optional request parameter, new endpoint, new enum value a client can ignore.
- Breaking: removing/renaming a field, changing a field's type, changing status code semantics, making an optional field required.
- A good habit: run new and old response shapes through the same test client to confirm the old contract still parses correctly.

2. URI Path Versioning

The most common and most discoverable strategy embeds the version directly in the URL path — `/api/v1/users`, `/api/v2/users`. It's simple to implement (route two versions to two different router modules), trivially visible in logs and browser history, and easy for API consumers to understand without reading documentation. Its downside is that a 'version' technically applies to the whole URL even when only one resource actually changed, and it can encourage duplicating large amounts of routing code between versions if not organized carefully.

JAVASCRIPT

```
const usersV1Router = require("./routes/v1/users");
const usersV2Router = require("./routes/v2/users");

app.use("/api/v1/users", usersV1Router);
app.use("/api/v2/users", usersV2Router);
```

3. Header-Based Versioning

An alternative keeps URLs version-free and instead reads the desired version from a request header — either a custom header (Api-Version: 2) or via HTTP content negotiation using the Accept header's media type (Accept: application/vnd.myapi.v2+json). This keeps URLs 'clean' and stable over time (useful if URLs are treated as permanent resource identifiers), but it's less discoverable — a developer can't tell the API version just by looking at a URL in a browser or a bug report, and it requires every client and every piece of tooling (caching proxies, API gateways) to correctly forward the custom header.

JAVASCRIPT

```
function versionRouter(req, res, next) {
  const version = req.headers["api-version"] === "2" ? "v2" : "v1";
  req.apiVersion = version;
  next();
}

app.use("/api/users", versionRouter, (req, res) => {
  const controller = req.apiVersion === "v2" ? usersControllerV2 :
  usersControllerV1;
  controller.list(req, res);
});
```

4. Query Parameter Versioning

The third approach passes the version as a query parameter (/api/users?version=2). It's easy to implement and test manually in a browser, but it's the weakest of the three options in practice: a client that forgets the parameter silently falls back to a default version rather than getting an explicit error, query parameters are more likely to be stripped or ignored by caching layers, and it mixes versioning concerns with filtering/sorting concerns in the same part of the URL. Most production APIs favor URI path or header versioning over this approach for anything beyond a quick prototype.

5. Choosing a Versioning Granularity

Versioning can apply to the whole API at once (a single /v1 vs /v2 covering every resource) or per-resource (only the orders resource has a v2, while everything else stays on v1). Whole-API versioning is simpler to reason about but forces a major version bump even when only one endpoint changed, and requires maintaining two entire copies of unrelated code. Per-resource versioning is more precise but harder for consumers to track, since 'the API version' stops being a single meaningful number. Most teams start with whole-API versioning for simplicity and only split to per-resource versioning if the maintenance burden of duplicating unrelated code becomes painful.

6. Deprecating and Retiring Old Versions

Introducing a new version is only half the job — an old version left running forever becomes a permanent maintenance burden and a security liability (bug fixes and security patches often can't be safely backported to it). A responsible deprecation process announces the deprecation with a clear timeline, adds a Deprecation and Sunset HTTP header (or an equivalent field in the response) to responses from the old version so automated tooling can detect it, and only removes the old version after that communicated date has passed and usage has been confirmed to have dropped to near zero via server-side logging.

- Communicate deprecation timelines well in advance (weeks to months, depending on your consumer base).
- Add a Sunset HTTP header (RFC 8594) or a Deprecation header to responses from the version being retired.
- Monitor server-side usage metrics per version before removing one — don't rely on consumers to self-report they've migrated.
- Keep a changelog documenting exactly what changed between versions.

Common Pitfalls

- Bumping the API version for a purely additive, backwards-compatible change (a new optional field) when no version bump was needed at all.
- Removing or renaming a response field without any version bump, silently breaking every client still parsing the old field name.
- Relying on query-parameter versioning, where a client that forgets the parameter silently gets a default version instead of an explicit error.
- Never actually retiring old versions, leading to an ever-growing number of parallel code paths that all need bug fixes and security patches.
- Failing to communicate a deprecation timeline before removing an old version, breaking consumers with no warning.
- Versioning the whole API when only one resource actually changed, forcing unrelated clients to migrate for no functional reason.

Key Takeaways

- Version only for breaking changes — additive changes (new optional fields/endpoints) don't require a new version.
- URI path versioning (/api/v1/...) is the most discoverable and widely used strategy; header-based versioning keeps URLs stable but is less visible.
- Query parameter versioning is the weakest option — it fails silently when a client omits it.
- A version needs a deprecation plan from the moment it launches, not just a launch plan — use Sunset/Deprecation headers and a communicated timeline.

- Choose whole-API vs. per-resource versioning based on how often different resources actually change independently.

Learn More

RESTful API — Versioning

<https://restfulapi.net/versioning/>

IETF RFC 8594 — The Sunset HTTP Header Field

<https://datatracker.ietf.org/doc/html/rfc8594>

Microsoft Learn — API Versioning Guidance

<https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design#versioning-a-restful-web-api>